

Optimal Spatial Design for Correlated Processes via Virtual Noise

Continuous Measure Optimization and Brimkulov Exact Design Heuristics

Werner Müller, Markus Hainy

2026-06-05

Contents

1. Theoretical Framework	1
2. Interactive Parameters and Configuration	4
3. Core Function Architecture	4
4. Empirical Spatial Parameter Estimation	8
5. Single Setup Evaluation ($n = 10$)	9
6. Multi-Sample Size Analysis	12

1. Theoretical Framework

Selecting optimal monitoring networks or experimental designs under spatial dependency is a difficult problem in computational statistics. Unlike classic experimental designs where observations are independent and information matrices scale linearly with design weights ($\sum \xi_i F_i F_i^\top$), spatial designs feature observation errors that are structurally **correlated**. This lack of linearity means standard convex optimization toolkits cannot be used directly. The **Virtual Noise (VN)** methodology solves this by regularizing the problem into a continuous design space.

1.1 The Underlying Spatial Model

We consider a spatial field where observations are gathered at a set of locations x . The response variable $y(x)$ is defined by the linear model:

$$y(x) = f^\top(x)\beta + \varepsilon(x)$$

Where:

- $f(x)$ represents a vector of spatial trend regression functions. In this notebook, it corresponds to polynomial expansions of order 0 (constant), 1 (linear trend), or 2 (quadratic surface) across two-dimensional spatial coordinates.
- β is the vector of unknown regression parameters.
- $\varepsilon(x)$ is a stationary Gaussian process with a mean of zero and a spatial covariance structure defined by an exponential kernel: $K(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|}{\alpha_{\text{Range}}}\right)$.

The joint covariance matrix depends heavily on the specific configuration of the selected design points. The information matrix for an exact design selecting a subset of points I (where $|I| = n$) from a candidate set $\mathcal{X}$ of size N is:

$$M(I) = F_I^T C_{I,I}^{-1} F_I$$

where F_I is the $n \times p$ design matrix of the mean trend and $C_{I,I}$ is the $n \times n$ covariance matrix of the errors corresponding to the chosen points. Because $C_{I,I}^{-1}$ does not decompose linearly into individual point contributions, traditional continuous design relaxation cannot be directly applied.

1.2 The Virtual Noise Formulation

To restore a convex optimization framework for correlated data, the **Virtual Noise (VN) Method** introduces an artificial, independent error component $u_\xi(x)$ at each candidate point $x \in \mathcal{X}$. The variance of this virtual noise is inversely proportional to the continuous design weight $\xi(x)$.

The design measure ξ is defined on the finite design space $\mathcal{X} = \{x_1, \dots, x_N\}$ and belongs to the restricted set:

$$\Xi^+ = \left\{ \xi : \sum_{i=1}^N \xi(x_i) = 1, \quad 0 \leq \xi(x_i) \leq \frac{1}{n} \right\}$$

Original Formulation (Pázman et al., 2022) In the original formulation, the variance of the virtual noise component $u_\xi(x_i)$ is expressed as:

$$\text{var}\{u_\xi(x_i)\} = \kappa \left(\frac{1}{n\xi(x_i)} - 1 \right)$$

where $\kappa > 0$ is a small tuning/regularization parameter. When $\xi(x_i) = 1/n$, the virtual noise vanishes completely, aligning the matrix exactly with an n -point exact design. If $\xi(x_i) \rightarrow 0$, the variance approaches infinity, effectively stripping that location of any predictive weight.

Modified Formulation (Hainy, Müller, & Pázman, 2025) The modified formulation scales the virtual noise by the localized process variance $\sigma^2(x_i) = C_{i,i}$ to improve numerical stability and establish a cleaner mathematical relationship with classical uncorrelated design measures:

$$\text{var}\{u_\xi(x_i)\} = \kappa \sigma^2(x_i) \left(\frac{1}{n\xi(x_i)} - 1 \right)$$

Under both formulations, the global virtual noise information matrix is computed across the full candidate space:

$$M(\xi) = F^T (C + W_\kappa(\xi))^{-1} F$$

where $W_\kappa(\xi)$ is a diagonal matrix containing the elements $\text{var}\{u_\xi(x_i)\}$.

Under the **Modified Virtual Noise Representation**, the information matrix can be rewritten as

$$M(\xi) = F_{\text{mod}}^T (K + W_\kappa(\xi))^{-1} F_{\text{mod}}$$

where $F_{\text{mod}} = \text{diag}(\sigma)^{-1} F$, $K = \text{cov2cor}(C)$ is the correlation matrix, and $W_\kappa(\xi)$ contains the virtual noise variances according to the original formulation. Therefore, the modified formulation corresponds to the original formulation with the design matrix and the covariance matrix replaced by their scaled versions.

For numerical stability, the information matrix can alternatively be computed in the following way:

$$M(\xi) = F^T Z(\xi)^{-1} \text{diag}(\xi) F$$

where

$$Z(\xi) = (C - \kappa I) \text{diag}(\xi) + \frac{\kappa}{n} I$$

1.3 Design Criteria and SLSQP Optimization

Designs are optimized based on specific functional objectives:

- **D-Optimality ("det"):** Maximizes the determinant of the information matrix, which minimizes the volume of the parameter confidence ellipsoid:

$$\Phi_D(\xi) = \log \det M(\xi)$$

- **A-Optimality ("trace"):** Minimizes the average variance of the parameter estimates, which corresponds to minimizing the trace of the inverse information matrix:

$$\Phi_A(\xi) = -\text{tr}(M(\xi)^{-1})$$

This notebook employs the **Sequential Least Squares Quadratic Programming (SLSQP)** algorithm from the `nloptr` package (`NLOPT_LD_SLSQP`). SLSQP is highly effective here because it natively handles:

- **Box constraints:** $0 \leq \xi_i \leq 1/n$
- **Linear equality constraints:** $\sum_{i=1}^N \xi_i = 1$

1.4 The Brimkulov Exchange Algorithm

To construct a realistic, exact n -point design from candidate sites, we use the heuristic **Brimkulov exchange algorithm**. This algorithm works sequentially:

1. It evaluates the spatial sensitivity of the current network profile.
2. It drops the point that contributes the least to the target design criterion.
3. It searches the remaining candidate sites to find and add the point that maximizes the design objective.

This exchange process continues iteratively until the structural efficiency updates converge.

1.5 Design Efficiency Calculations

The continuous virtual noise information matrix provides an upper bound for the performance of any exact design. We quantify the quality of our exact design using **Design Efficiency**:

- **D-Efficiency (crit = "det"):**

$$\text{Eff}_D = \left(\frac{\det(M_{\text{exact}})}{\det(M(\xi^*))} \right)^{1/p} = \exp \left(\frac{\Phi_{\text{exact}} - \Phi_{\text{virtual noise}}}{p} \right)$$

- **A-Efficiency (crit = "trace"):**

$$\text{Eff}_A = \frac{\text{tr}(M(\xi^*)^{-1})}{\text{tr}(M_{\text{exact}}^{-1})} = \frac{\Phi_{\text{virtual noise}}}{\Phi_{\text{exact}}}$$

where p represents the number of linear parameters in the mean function.

2. Interactive Parameters and Configuration

This section defines the operational options for the notebook. You can modify these variables to change the design criteria, polynomial complexity, or sample sizes across the entire analysis.

```
# Define your workspace options here:
n      <- params$n      # Number of target exact points to choose
crit   <- params$crit   # Selection criterion: "det" (D-optimal) or "trace" (A-optimal)
# Polynomial order of spatial coordinates: 0 = Constant, 1 = Linear, 2 = Quadratic
poly.order <- params$poly.order
modVN   <- params$modVN # Virtual Noise Representation: TRUE = Modified, FALSE = Original

# Define paths to data files
shapefile_path <- "oogemeinden.shp"
csv_path      <- "ua-stations.csv"

library(nloptr)    # For non-linear programming via SLSQP
library(tidyverse) # For data manipulation (dplyr, purrr, readr) and plotting (ggplot2)
library(sf)        # For handling spatial simple features (shapefiles)
library(fields)    # For spatial process models and Kriging parameter estimation
library(parallel)  # For parallel processing across multiple CPU cores
```

3. Core Function Architecture

The code modules from the workshop scripts (`criterion_functions.r`, `Brimkulov_functions.r`, and `optim_functions.r`) are embedded directly below to ensure the notebook is fully self-contained.

3.1 Criterion and Gradient Evaluation Engine

```
# Compute efficiency metrics relative to continuous virtual noise bounds
compute_efficiency <- function(crit.exact, crit.bound, crit, p) {
  if (crit == "det") {
    exp((crit.exact - crit.bound) / p)
  } else {
    return(crit.bound / crit.exact)
  }
}

# Helper function for floor rounding to given significant digits
down_signif <- function(x, digits = 0) {
  m <- 10^(ceiling(log(x, 10)) - digits)
  (x %/% m) * m
}

# Calculate performance criterion under Virtual Noise representations
compute_criterion <- function(xi.vec, n, model.comps, crit = "det") {
  F <- model.comps$F
  K <- model.comps$K
  kappa <- model.comps$kappa

  xi.vec <- as.vector(xi.vec)
  N <- length(xi.vec)
  Z <- (K - diag(kappa, nrow = N, ncol = N)) * xi.vec + diag(kappa/n, nrow = N, ncol = N)
```

```

Zinv <- solve(Z)
ZinvT.F <- crossprod(Zinv, F)
M <- crossprod(ZinvT.F, F * xi.vec)

if (crit == "det") {
  detM <- det(M)
  return(log(detM))
} else {
  Minv <- chol2inv(chol(M))
  return(-sum(diag(Minv)))
}
}

# Compute gradient vectors for the continuous criterion function
criterion.gradient <- function(xi.vec, n, model.comps, crit = "det") {
  F <- model.comps$F
  K <- model.comps$K
  kappa <- model.comps$kappa

  xi.vec <- as.vector(xi.vec)
  N <- length(xi.vec)
  Z <- (K - diag(kappa, nrow = N, ncol = N)) * xi.vec + diag(kappa/n, nrow = N, ncol = N)
  Zinv <- solve(Z)
  ZinvT.F <- crossprod(Zinv, F)
  M <- crossprod(ZinvT.F, F * xi.vec)
  Minv <- chol2inv(chol(M))

  if (crit == "det") {
    Phi <- log(det(M))
    DeltaPhi <- (kappa/n) * apply(ZinvT.F, 1, function(l) l %*% Minv %*% l)
  } else {
    Minv_sq <- Minv %*% Minv
    trace_Minv <- sum(diag(Minv))
    Phi <- -trace_Minv
    DeltaPhi <- (kappa/n) * apply(ZinvT.F, 1, function(l) l %*% Minv_sq %*% l)
  }

  return(list(Phi = Phi, DeltaPhi = DeltaPhi))
}

```

3.2 Optimization Matrix Generators

```

# Construct components: design matrices, covariance/correlation arrays, and minimum eigenvalues
get.model.comps <- function(grid, kernelfuns, poly.order = 1, modVN = FALSE) {
  N <- dim(grid)[1]
  kernelfun <- kernelfuns[[poly.order + 1]]

  K <- matrix(nrow = N, ncol = N)
  for (i in 1:N) {
    for (j in 1:N) {
      K[i, j] <- kernelfun(grid[i, ], grid[j, ])
    }
  }
}

```

```

if (poly.order == 0) {
  F <- matrix(rep(1, N), ncol = 1)
} else if (poly.order == 1) {
  F <- cbind(1, grid)
} else if (poly.order == 2) {
  F <- cbind(1, grid, grid^2, grid[, 1] * grid[, 2])
}

if (modVN) {
  sigma <- sqrt(diag(K))
  K <- cov2cor(K)
  F <- F / sigma
}

kappa <- down_signif(min(eigen(K)$values), digits = 4)
return(list(F = F, K = K, kappa = kappa, grid = grid))
}

```

3.3 Continuous SLSQP and Discrete Exact Optimizers

```

# Continuous Optimization using SLSQP
find.opt.measure <- function(n, poly.order, crit, modVN, kernelfuns, grid) {
  model.comps <- get.model.comps(grid, kernelfuns, poly.order, modVN)

  critfun <- function(xi) -compute.criterion(xi, n, model.comps, crit)
  critgrad <- function(xi) -criterion.gradient(xi, n, model.comps, crit)$DeltaPhi
  equ.func <- function(xi) sum(xi) - 1
  equ.jac <- function(xi) matrix(rep(1, length(xi)), nrow = 1)

  xi <- rep(1 / nrow(grid), nrow(grid))

  ret <- nloptr(xi, eval_f = critfun, eval_grad_f = critgrad, eval_g_eq = equ.func,
    eval_jac_g_eq = equ.jac,
    lb = rep(0, length(xi)), ub = rep(1 / n, length(xi)),
    opts = list(algorithm = "NLOPT_LD_SLSQP", xtol_rel = 1e-4, maxeval = 5000))

  return(list(design.measure = ret$solution, crit.bound = -ret$objective))
}

# Calculate criteria evaluations directly for exact configurations
compute.criterion.exact <- function(selected, model.comps, crit) {
  F.full <- model.comps$F
  K.full <- model.comps$K

  K <- K.full[selected, selected, drop = FALSE]
  Kinv <- chol2inv(chol(K))
  F <- F.full[selected, , drop = FALSE]
  M <- crossprod(F, Kinv %*% F)

  if (crit == "det") {
    return(log(det(M)))
  } else {
    Minv <- chol2inv(chol(M))

```

```

    return(-sum(diag(Minv)))
  }
}

# Compute localized sensitivity metrics for dependent data structures
compute.sensitivity <- function(candidate, selected, crit, F.full, K.full, reduced.model.comps = NULL)
  if (is.null(reduced.model.comps)) {
    K <- K.full[selected, selected, drop = FALSE]
    Kinv <- chol2inv(chol(K))
    F <- F.full[selected, , drop = FALSE]
    M <- crossprod(F, Kinv %*% F)
    Minv <- chol2inv(chol(M))
  } else {
    Kinv <- reduced.model.comps$Kinv
    F <- reduced.model.comps$F
    Minv <- reduced.model.comps$Minv
  }

  Fx <- F.full[candidate, , drop = FALSE]
  Kx <- K.full[selected, candidate]

  F.tilda <- t(Fx) - crossprod(F, Kinv %*% Kx)
  sigmasq_tilda <- K.full[candidate, candidate] - Kx %*% Kinv %*% Kx

  if (crit == "det") {
    return(as.vector(1 + crossprod(F.tilda, Minv %*% F.tilda) / sigmasq_tilda))
  } else {
    Minv.Ftilda <- Minv %*% F.tilda
    return(as.vector(sigmasq_tilda * crossprod(Minv.Ftilda) /
      (1 + sigmasq_tilda * crossprod(F.tilda, Minv.Ftilda))))
  }
}

# Core Brimkulov Exchange Engine
brimkulov <- function(n, model.comps, crit = "det") {
  F.full <- model.comps$F
  K.full <- model.comps$K
  grid <- model.comps$grid
  N <- dim(grid)[1]

  delta <- 0
  selected <- sort(sample(N, n, replace = FALSE))
  abort <- FALSE

  while (!abort) {
    sens.drop.vec <- numeric(n)
    for (i in seq_len(n)) {
      sens.drop.vec[i] <- compute.sensitivity(selected[i], selected[-i], crit = crit, F.full, K.full)
    }
    sens.drop <- min(sens.drop.vec)
    drop.index <- which.min(sens.drop.vec)
    selected.reduced <- selected[-drop.index]
  }
}

```

```

K <- K.full[selected.reduced, selected.reduced, drop = FALSE]
Kinv <- chol2inv(chol(K))
F <- F.full[selected.reduced, , drop = FALSE]
M <- crossprod(F, Kinv %*% F)
Minv <- chol2inv(chol(M))

candidates <- setdiff(1:N, selected.reduced)
reduced.model.comps <- list(F = F, Kinv = Kinv, Minv = Minv)
sens.add.vec <- sapply(candidates, FUN = compute.sensitivity, selected = selected.reduced,
                      crit = crit, F.full, K.full, reduced.model.comps)
sens.add <- max(sens.add.vec)

if (sens.add - sens.drop > delta) {
  add.index <- which.max(sens.add.vec)
  selected <- c(selected.reduced, candidates[add.index])
  selected <- sort(selected)
} else {
  abort = TRUE
}
}

design <- grid[selected, , drop = FALSE]
return(list(design = design, selected = selected))
}

# Exact Optimization Heuristic Controller
find.opt.exact <- function(n, poly.order, crit, kernelfuns, grid, nruns = 100) {
  model.comps <- get.model.comps(grid, kernelfuns, poly.order, modVN = FALSE)
  exact.designs <- list()
  crit.exact.vec <- numeric(length = nruns)

  for (i in 1:nruns) {
    retbrim <- brimkulov(n, model.comps, crit)
    exact.designs[[i]] <- retbrim$design
    crit.exact.vec[i] <- compute.criterion.exact(retbrim$selected, model.comps, crit)
  }

  maxindex <- which.max(crit.exact.vec)
  return(list(opt.design = exact.designs[[maxindex]], crit.exact = crit.exact.vec[maxindex]))
}

```

4. Empirical Spatial Parameter Estimation

We load the spatial geometries and meteorological data profiles to estimate the process variance (σ^2) and range parameters (a_{Range}) across the different trend specifications.

```

# Ingest Shapefile and Station Records
boundaries <- st_read(shapefile_path, quiet = TRUE)
stations_wide <- read_csv2(csv_path)

stations <- stations_wide |>
  select(name, stationnumber, longitude, latitude, `199407`) |>

```



```

rename(rainfall = `199407`) |>
filter(!is.na(rainfall))

# Compute centroids from geometry shapes to establish our prediction grid
cent <- st_centroid(boundaries$geometry)
long_centr <- map_dbl(cent, 1)
lat_centr <- map_dbl(cent, 2)

municip_coord <- as_tibble(boundaries) |>
  select(c(LBBGG, GEM_NAME16, GEM_NAME)) |>
  add_column(LONG = long_centr, LAT = lat_centr)

grid <- as.matrix(municip_coord[c("LONG", "LAT")])
coord <- matrix(c(stations$longitude, stations$latitude), ncol = 2)
rainfall <- stations$rainfall

```

Using `fields::spatialProcess`, we model the exponential covariance parameter targets based on the order of the mean polynomial trend.

```

get.parameters <- function(poly.order){
  ua_fit <- spatialProcess(x = coord, y = rainfall,
    mKrig.args = list(m = poly.order + 1),
    cov.args = list(Covariance = "Exponential"))
  sigma2 <- unname(ua_fit$summary["sigma2"])
  aRange <- unname(ua_fit$summary["aRange"])
  return(list(sigma2 = sigma2, aRange = aRange))
}

get.kernelfun <- function(pars) {
  sigma2 <- pars$sigma2
  aRange <- pars$aRange
  function(x, y) {
    sigma2 * exp(-sqrt(sum((x - y)^2)) / aRange)
  }
}

pars <- map(0:2, get.parameters)
kernelfuns <- map(pars, get.kernelfun)

```

Below are the estimated covariance parameter values for each trend specification:

Trend	Sigma2	aRange
Constant (Order 0)	1518.1403	33226.868
Linear (Order 1)	1439.6742	31603.714
Quadratic (Order 2)	417.5374	5462.511

5. Single Setup Evaluation ($n = 10$)

We perform the continuous and exact optimization routines for the specified system configuration:

- n : 10
- criterion: det

- polynomial order: 2
- modified VN: TRUE

```
model.comps <- get.model.comps(grid, kernelfuns, poly.order, modVN)
p <- ncol(model.comps$F)

# Continuous Optimization (Virtual Noise SLSQP Boundary)
ret.measure <- find.opt.measure(n, poly.order, crit, modVN, kernelfuns, grid)

# Heuristic Exact Optimization (Brimkulov Exchange Network)
ret.design <- find.opt.exact(n, poly.order, crit, kernelfuns, grid, nruns = 40)

# Compute and display the resulting design efficiency
eff_single <- compute.efficiency(ret.design$crit.exact, ret.measure$crit.bound, crit, p)
```

Optimization Performance: * Continuous Virtual Noise Bound Value: **152.5525** * Exact Brimkulov Design Criterion Value: **147.8367** * Resulting Spatial Network Efficiency: **45.57%**

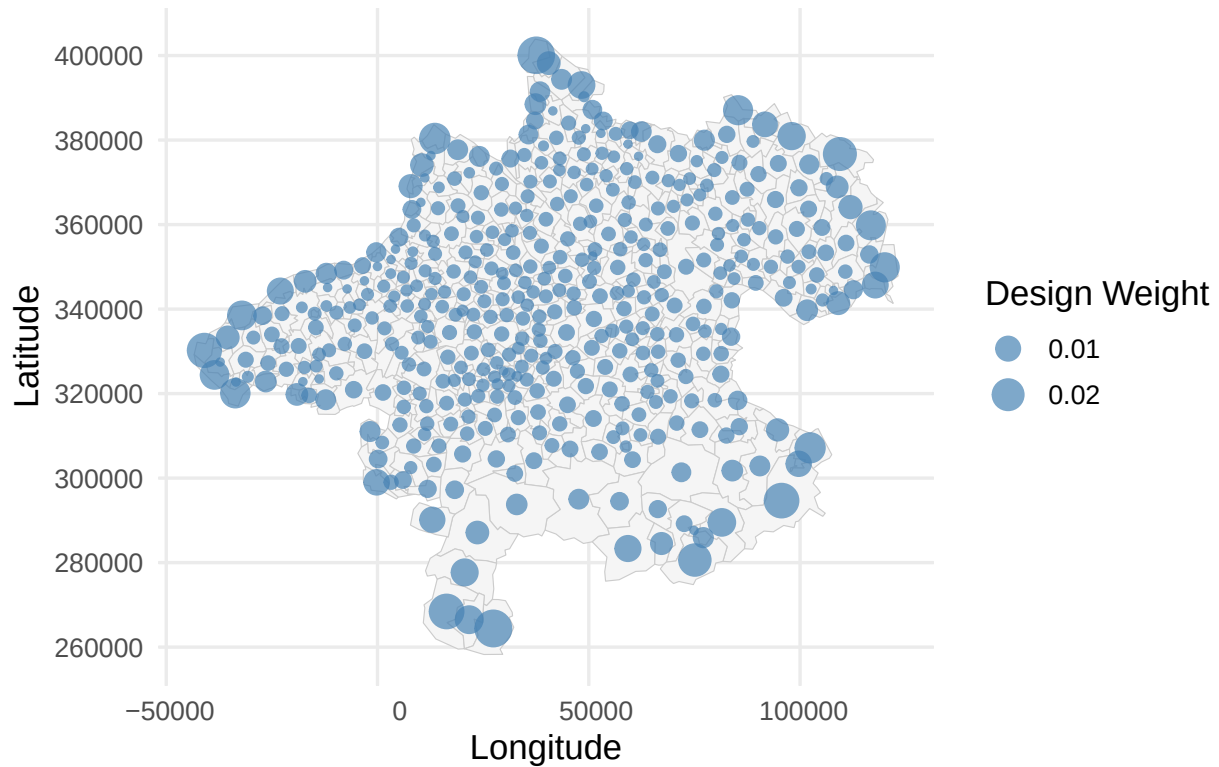
5.1 Continuous Measure Density Map

This map shows the continuous weight allocation across the candidate locations. Larger point sizes indicate locations where the optimization places higher value.

```
plotdata <- as.data.frame(grid) |> add_column(measure = ret.measure$design.measure)

ggplot(plotdata) +
  geom_sf(data = boundaries, fill = "gray96", color = "gray80") +
  geom_point(mapping = aes(x = LONG, y = LAT, size = measure), color = "steelblue", alpha = 0.7) +
  labs(x = "Longitude", y = "Latitude", size = "Design Weight",
       title = "Continuous Virtual Noise Weight Allocations") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "right")
```

Continuous Virtual Noise Weight Allocations



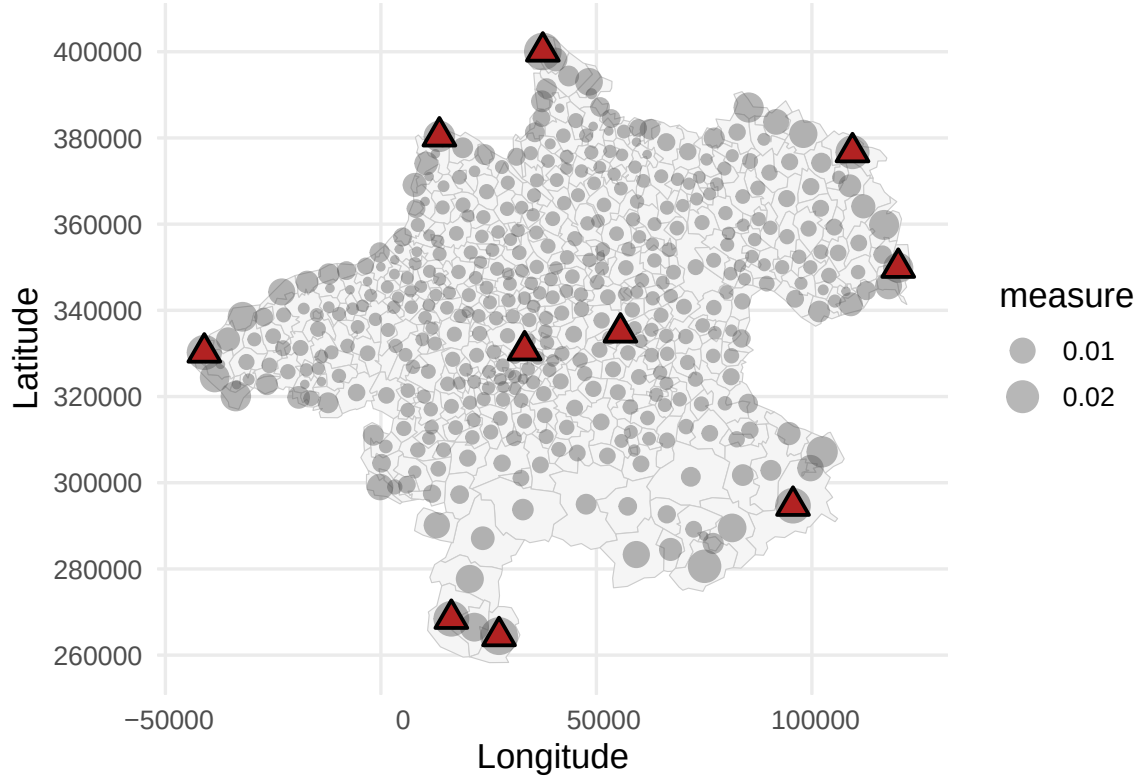
5.2 Heuristic Exact Allocation Overlay

This visualization overlays the exact n -point network chosen by the Brimkulov exchange algorithm (shown as red triangles) on top of the continuous continuous weight densities.

```
plotdata.brim <- as.data.frame(ret.design$opt.design)

ggplot(plotdata) +
  geom_sf(data = boundaries, fill = "gray96", color = "gray80") +
  geom_point(mapping = aes(x = LONG, y = LAT, size = measure), alpha = 0.4, color = "grey30") +
  geom_point(mapping = aes(x = LONG, y = LAT), data = plotdata.brim,
    shape = 24, fill = "firebrick", color = "black", size = 3.5, stroke = 1) +
  labs(x = "Longitude", y = "Latitude", title = "Brimkulov Selected Points vs. Continuous Weights") +
  theme_minimal(base_size = 13)
```

Brimkulov Selected Points vs. Continuous Weights



6. Multi-Sample Size Analysis

To understand how network scale affects performance, we evaluate how design efficiency changes across a range of sample sizes n . This loop can require significant computation time, so we use parallel processing across multiple CPU cores to speed up the calculations. The following code is only evaluated if the parameter `eval_multi_n` in the YAML header is set to `TRUE`.

```
# Define the range of sample sizes based on the number of parameters
n.range <- switch(poly.order + 1, 2:16, 4:16, 7:16)

# Set up the parallel execution cluster environment
ncores <- min(detectCores(logical = FALSE), 4) # Limit to 4 cores for safe compilation
cl <- makePSOCKcluster(ncores)

# Export essential functions and objects to the worker nodes
clusterExport(cl, c("grid", "kernelFUNs", "get.model.comps", "compute.criterion",
                    "criterion.gradient", "compute.criterion.exact", "compute.sensitivity",
                    "brimkulov", "down_signif", "find.opt.measure", "find.opt.exact", "modVN"))

# Initialize nloptr library on all workers
clusterEvalQ(cl, library(nloptr))

## [[1]]
## [1] "nloptr"      "stats"      "graphics"   "grDevices"  "utils"      "datasets"
## [7] "methods"    "base"
```

```
##
## [[2]]
## [1] "nloptr"      "stats"      "graphics"   "grDevices" "utils"      "datasets"
## [7] "methods"    "base"
##
## [[3]]
## [1] "nloptr"      "stats"      "graphics"   "grDevices" "utils"      "datasets"
## [7] "methods"    "base"
##
## [[4]]
## [1] "nloptr"      "stats"      "graphics"   "grDevices" "utils"      "datasets"
## [7] "methods"    "base"

# Perform the calculations across the workers
measure.list <- parLapplyLB(cl, n.range, find.opt.measure, poly.order = poly.order,
                           crit = crit, modVN = modVN, kernelfuns = kernelfuns, grid = grid)

design.list <- parLapplyLB(cl, n.range, find.opt.exact, poly.order = poly.order,
                          crit = crit, kernelfuns = kernelfuns, grid = grid, nruns = 20)

# Stop the cluster safely
stopCluster(cl)

# Compile results and calculate efficiencies
crit.bounds <- map_dbl(measure.list, "crit.bound")
crit.vals <- map_dbl(design.list, "crit.exact")
efficiencies <- map2_dbl(crit.vals, crit.bounds, compute.efficiency, crit = crit, p = p)
names(efficiencies) <- n.range
```

6.1 Efficiency Tracking Across Sample Sizes

The table and plot below track how closely the heuristic exact n -point designs approach the continuous virtual noise upper bound across different sample sizes.

Network Size (n)	Continuous Bound	Exact Criterion	Design Efficiency
7	151.3831	146.0356	41.01%
8	151.8377	146.7024	42.49%
9	152.2222	147.3007	44.03%
10	152.5525	147.8395	45.59%
11	152.8399	148.3628	47.42%
12	153.0927	148.8029	48.92%
13	153.3169	149.2158	50.48%
14	153.5175	149.5888	51.96%
15	153.6980	149.9302	53.37%
16	153.8614	150.2232	54.53%

```
plot(x = n.range, y = efficiencies, type = "o", ylim = c(0, 1.1),
     pch = 16, col = "darkblue", lwd = 2, las = 1,
     xlab = "Network Configuration Size (n)", ylab = "Design Efficiency",
     main = paste0(ifelse(crit == "det", "D", "A"), "-Optimal Exact Design Efficiency Path"))
abline(h = 1.0, lty = "dotted", col = "firebrick", lwd = 1.5)
grid()
```

D-Optimal Exact Design Efficiency Path

